

SR NEXT

Self-Guided Internship Program

Phase 2 Report

AI-Powered Instagram Influencer Analytics & Automation Insights

Student Name	SHRE RAAM P J
Role	Independent Intern / Project Contributor
Program	SR NEXT Self-Guided Internship
Phase	Phase 2 — AI-Powered Analysis & Ranking
Mode	Remote / Self-Guided
Date	May 2026

1. Objective

The objective of Phase 2 was to take the raw Instagram influencer dataset collected during Phase 1 and transform it into a fully analysed, ranked, and visualised output using Python, data engineering techniques, natural language processing, machine learning models, and interactive dashboards.

Phase 2 built directly on the 2,093 influencer records collected in Phase 1, addressing the data quality issues identified during collection and adding new analytical layers to produce portfolio-ready deliverables.

2. Work Completed

During Phase 2, the following major deliverables were produced:

- A fully cleaned and structured dataset with all data quality issues resolved
- A composite Influence Score formula (0-100) combining 4 weighted features
- A ranked influencer list with tier classification across 4 levels
- An automation likelihood detection system using pattern-based analysis
- Two machine learning models: a Random Forest Classifier and a Random Forest Regressor
- A 5-sheet formatted Excel output with colour-coded rankings
- A standalone interactive HTML dashboard with 6 pages
- A live Streamlit web dashboard with real-time sidebar filters
- An end-to-end pipeline script that runs the entire project with one command

3. Process & Workflow

3.1 Data Cleaning

The raw dataset from Phase 1 contained several data quality issues that prevented direct analysis. Three critical problems were identified and resolved using custom Python parsing functions.

Followers Column

The follower counts were stored in mixed formats across the dataset. Some rows contained plain integers such as 348724, while others contained shorthand text such as 244.9K or 1.5M. Python's standard `pd.to_numeric()` function could not parse these text formats and converted them to NaN, causing those rows to show zero followers. A custom `parse_followers()` function was written to detect the format of each value and convert accordingly — multiplying by 1,000 for K values and 1,000,000 for M values, then converting all results to plain integers.

Engagement Rate Column

The engagement rate column contained mixed formats including plain decimals (0.0267), percentage strings (5.2%), and comparison operators (<0.1%, >20%). Standard numeric conversion failed on the text-based formats. A custom `parse_er()` function was written to strip the percentage symbol, less-than and greater-than symbols, then convert to decimal form. Values containing a percentage symbol or a numeric value greater than 1 were divided by 100 to standardise to decimal representation.

Posting Frequency Column

The posting frequency column contained descriptive text such as ~20-30/month, 3 posts/week, and Daily rather than numeric values. A custom `parse_freq()` function was written to handle each format: ranges were averaged (20-30 became 25.0), weekly values were multiplied by 4.3 to convert to monthly equivalents, and the word Daily was mapped to 30 posts per month.

3.2 Feature Engineering

After cleaning, four new features were engineered from the existing columns to enable scoring and ML training.

- **Hashtag Count:** The Hashtags column was parsed using a regular expression to count the number of distinct hashtags per account. This produced a numeric indicator of content strategy effort.
- **Broad Category:** The original dataset contained over 500 unique category labels written inconsistently. A keyword-based NLP classification function was written to group these into 13 broad categories including Fashion & Beauty, Health & Fitness, Music, Travel, Technology & Gaming, and others. This technique is called rule-based text classification, which is a form of NLP.
- **Normalised Scores:** All four features used in the Influence Score were normalised to a 0-1 scale to allow fair comparison. Followers were log-scaled to reduce the disproportionate impact of very large accounts.
- **Posts Per Month:** The parsed posting frequency values were stored as a clean numeric column for use in scoring and ML models.

3.3 Composite Influence Score

A composite Influence Score was designed to rank influencers on a single 0-100 scale by combining four normalised features with weighted contributions:

Feature	Weight	Justification
Engagement Rate	40%	Most important metric — measures real audience response and content quality
Posting Frequency	25%	Consistency indicates an active, serious influencer

Followers (log-scaled)	20%	Reach matters but log scaling reduces unfair advantage of mega-accounts
Hashtag Diversity	15%	Minor signal of content strategy and discoverability effort

Formula: Influence Score = (ER x 0.40) + (Frequency x 0.25) + (Followers x 0.20) + (Hashtags x 0.15) x 100

All features were normalised to 0-1 before applying weights. The final score was multiplied by 100 to produce an intuitive 0-100 scale. Influencers were then sorted by this score and assigned ranks 1 to 2,093.

3.4 Tier Classification

Influencers were grouped into four tiers based on their rank percentile:

- Tier 1 — Top 10%: 209 influencers — priority outreach targets
- Tier 2 — Top 30%: 418 influencers — strong secondary candidates
- Tier 3 — Mid 60%: 630 influencers — average performers
- Tier 4 — Lower 40%: 836 influencers — lower engagement or activity

3.5 Automation Likelihood Detection

An automation detection system was built to identify accounts likely using bots, schedulers, or engagement-buying services. The system analyses two key patterns in the data:

High automation risk accounts satisfy one of two conditions: an engagement rate greater than 30% on an account with more than 50,000 followers, which is statistically impossible through organic means at that scale, or a posting frequency greater than 25 times per month combined with an engagement rate below 2%, which indicates a bot scheduler posting content with no real audience response.

Medium automation risk accounts show an engagement rate above 20% on any account size, or a posting frequency above 20 per month combined with below-average engagement below 4%.

Risk Level	Condition	Count in Dataset
High	ER > 30% on >50K account OR posts >25/month with ER < 2%	196 accounts (9.4%)
Medium	ER > 20% on any account OR high freq with ER < 4%	169 accounts (8.1%)

Low	Normal organic engagement patterns	1,728 accounts (82.6%)
-----	------------------------------------	------------------------

4. Machine Learning Models

4.1 Model 1 — Automation Likelihood Classifier

A Random Forest Classifier was trained to predict automation likelihood (High, Medium, or Low) from five input features: Followers, Engagement Rate, Posts Per Month, Hashtag Count, and Category (label-encoded).

The dataset was split 80% for training and 20% for testing. The model used 100 decision trees with a maximum depth of 10 and class_weight='balanced' to handle the imbalanced class distribution where Low risk accounts vastly outnumber High risk ones.

Metric	Result
Test Set Accuracy	100.00%
Cross-Validation Accuracy (5-fold)	94.75%
Top Feature	Engagement Rate (38% importance)
Second Feature	Posts Per Month (34% importance)

The 100% test accuracy is explained by the fact that the model learned the exact rule-based logic used to generate the labels. The 94.75% cross-validation score is a more realistic indicator of true generalisation performance.

4.2 Model 2 — Engagement Rate Predictor

A Random Forest Regressor was trained to predict engagement rate from the same five input features. A Linear Regression model was also trained for comparison. Outliers above the 99th percentile were excluded from training to improve model stability.

Metric	Random Forest	Linear Regression
R2 Score	0.9999	Lower
MAE	0.21%	Higher
Winner	Random Forest	-

5. Tools & Technologies Used

Tool / Library	Purpose
Python	Core programming language for all analysis and pipeline scripts
Pandas	Data loading, cleaning, transformation, and CSV/Excel export
NumPy	Numerical operations — log scaling, normalisation, array handling
scikit-learn	Random Forest Classifier, Random Forest Regressor, train/test split, cross-validation, metrics
Matplotlib / Seaborn	Confusion matrix plots and feature importance charts
Plotly	Interactive charts in HTML dashboard (pie, bar, scatter)
Streamlit	Live interactive web dashboard with real-time sidebar filters
openpyxl	Writing formatted multi-sheet Excel output with colours and styles
GitHub	Version control — all progress committed and pushed regularly

6. Challenges Faced & How They Were Resolved

Challenge 1 — Mixed Follower Count Formats

The follower column contained values in three different formats: plain integers, K shorthand (244.9K), and M shorthand (1.5M). Standard pandas numeric conversion treated K and M values as non-numeric, converting them to NaN which then became zero. This caused over 1,000 influencer records to show incorrect follower counts of zero.

Resolution: A custom parsing function was written using Python string matching and regular expressions. The function identifies the unit suffix (K or M), strips it, converts the number to float, and multiplies by the appropriate factor. All values were then stored as clean integers.

Challenge 2 — Engagement Rate Stored as Text

Engagement rates were stored inconsistently — some rows had decimal values like 0.0267, others had percentage strings like 5.2%, and some had comparison operators like <0.1% or >20%. These mixed formats prevented any calculation or normalisation.

Resolution: A `parse_er()` function was written to strip all non-numeric symbols, detect whether the value was in percentage format, and convert to a consistent decimal representation.

Challenge 3 — Over 500 Inconsistent Category Names

The Category column contained over 500 unique values for what were essentially the same niches — Fashion, Fashion & Lifestyle, Fashionista, and Style & Fashion all referred to the same content type. This made category-level analysis and ML encoding impossible without grouping.

Resolution: A keyword-based NLP classification function was written to map all 500+ category names into 13 broad groups based on keyword matching. This is a rule-based NLP technique that proved highly effective for this structured data.

Challenge 4 — Windows PATH Issue with Streamlit

When Streamlit was installed on Windows, the streamlit.exe command was placed in a Scripts directory that was not included in the system PATH variable. Running streamlit run in the terminal returned a CommandNotFoundException error.

Resolution: The correct way to run Streamlit on Windows without modifying PATH is to use `python -m streamlit run streamlit_app.py`, which bypasses the PATH lookup and runs Streamlit directly through the Python module system.

Challenge 5 — Streamlit Crash on Empty Filter Combinations

When sidebar filters were combined in ways that returned zero matching influencers, the application crashed with a ValueError because functions like `median()` and `mean()` return NaN on empty DataFrames, and `int(NaN)` raises an exception.

Resolution: An empty state check was added before all KPI metric calculations. When zero influencers match the current filters, the app displays a clean warning message rather than crashing.

Challenge 6 — Pipeline Reading Formatted Excel Instead of Raw Data

The end-to-end pipeline script initially read the `Final_phase_2.xlsx` file from the data folder, which is the formatted styled output with a title banner row as the first row. Pandas read the title as the column headers, resulting in columns named `Unnamed: 1`, `Unnamed: 2`, and so on instead of the actual field names.

Resolution: The pipeline was updated to prioritise raw input files (`cleaned_influencers_sorted.xlsx`, `Final_phase_1.xlsx`) over formatted outputs. A fallback mechanism was also added to automatically detect and skip title rows when reading formatted Excel files.

7. Key Results

Metric	Value
Total Influencers Analysed	2,093
Average Influence Score	33.00 / 100
Average Engagement Rate	39.65%
Median Followers	39,212

Tier 1 — Top 10%	209 influencers
High Automation Risk	196 accounts (9.4%)
Influencers with Contact Info	1,597 (76.3%)
Broad Categories	13 (from 501 original labels)
Top Ranked Influencer	studio_choom (Score: 91.10)
Pipeline Execution Time	4 seconds end-to-end

8. Deliverables Submitted

- final_phase_2.xlsx — 5-sheet formatted Excel with ranked influencers, category analysis, summary stats, automation risk, and legend
- SR_NEXT_Phase2_Ranked_Influencers.csv — Clean ranked CSV with all engineered features and ML predictions
- SR_NEXT_Phase2_Code.py — Full data cleaning, feature engineering, scoring, and ranking pipeline
- SR_NEXT_Phase2_ML_Models.py — Random Forest Classifier and Regressor with evaluation charts
- SR_NEXT_Phase2_Dashboard.html — Standalone interactive Plotly dashboard (6 pages, no installation required)
- streamlit_app.py — Live Streamlit dashboard with real-time filters and download buttons
- run_pipeline.py — End-to-end master pipeline script (one command runs the full project)
- Phase 2 Report — This document
- GitHub Repository — All code, data, images, and reports version-controlled at github.com/SHRE-RAAM-P-J

9. Conclusion

Phase 2 successfully transformed the raw 2,093-influencer dataset from Phase 1 into a complete AI-powered analytics system. The work covered the full data science workflow from raw messy data to a production-ready pipeline with ML models and interactive visualisations.

The most significant technical contributions were the custom data parsing pipeline that handled three different follower and engagement rate formats, the NLP-based category classification that reduced 501 unique labels to 13 meaningful groups, and the composite Influence Score formula that provides a single fair ranking across all influencers regardless of follower size.

The automation detection system identified 196 accounts with suspicious engagement patterns, providing actionable intelligence for brand safety decisions. Both ML models were successfully

trained and evaluated, with the Random Forest Classifier achieving 94.75% cross-validation accuracy.

The end-to-end pipeline, live Streamlit dashboard, and structured GitHub repository demonstrate production-level thinking in organising and delivering a data project. This phase established a strong foundation for real-world influencer marketing analytics.